Assignment 3: Database Design Analysis

DBMS Assignment Series | Project topic: **Online Food Delivery System (QuickBite)**

Name: **Jagan S** | Roll No.: **2501121240031**

1. Objective

Examine the database design of the Online Food Delivery System for redundancy and insertion, deletion and update anomalies, identify functional dependencies, explain how a poor design causes anomalies, and propose improvements.

2. The design being analysed

A beginner's first attempt at a food-delivery database often puts everything into **one flat table**, `ORDER_RECORD`, with one row for every dish in every order:

```
ORDER_RECORD(order_id, order_date, status, payment_method, delivery_fee,
             customer_id, customer_name, customer_email, customer_phone, customer_area, membership,
             restaurant_id, restaurant_name, cuisine, restaurant_area, rating,
             partner_id, partner_name, partner_phone, vehicle,
             item_id, item_name, category, item_price, quantity)
```

Primary key: **(order_id, item_id)**, because an order has many dishes. The table below shows real rows of this flat design for one customer (generated from the sample data); only some columns are shown:

order_id	customer	area	restaurant_name	item_name	quantity	partner_name	status
1	Aditya Rao	Indiranagar	Spice Garden	Paneer Tikka	1	Ravi Kumar	Delivered
1	Aditya Rao	Indiranagar	Spice Garden	Butter Chicken	1	Ravi Kumar	Delivered
1	Aditya Rao	Indiranagar	Spice Garden	Gulab Jamun	2	Ravi Kumar	Delivered
4	Aditya Rao	Indiranagar	Dragon Bowl	Veg Spring Rolls	1	Ravi Kumar	Delivered
4	Aditya Rao	Indiranagar	Dragon Bowl	Hakka Noodles	2	Ravi Kumar	Delivered
4	Aditya Rao	Indiranagar	Dragon Bowl	Lemon Iced Tea	2	Ravi Kumar	Delivered
11	Aditya Rao	Indiranagar	Dosa Corner	Masala Dosa	2	Ravi Kumar	Delivered

3. Data redundancy identified

- **Customer details repeated.** Name, e-mail, phone, area and membership are copied into every dish row of every order. The sample data has 52 order lines but only 11 customers who ordered, so customer data would be stored 52 times instead of 11.
- **Order details repeated.** Date, status, payment method and delivery fee are repeated for each dish of the same order (order 1 has 3 dishes, so 3 copies).
- **Restaurant, partner and dish details repeated.** 'Spice Garden / North Indian / 4.4' is copied for every dish ordered from it; a partner's name, phone and vehicle for every dish of every delivery; a dish's name, category and price every time it is ordered (22 distinct dishes were ordered 52 times).
- **Consequences:** wasted storage, slower updates, and a high risk of inconsistent copies (e.g. a restaurant rated 4.4 in some rows and 4.6 in others).

4. Anomalies

4.1 Insertion anomalies

- A new dish (e.g. 'Chicken Chettinad' or 'Mango Smoothie', which nobody has ordered yet) cannot be stored, because every row needs an order_id.
- A newly registered customer (Leela Pillai in our data) cannot be saved until she places an order.
- A new restaurant or a newly hired delivery partner (e.g. an offline partner with no deliveries) cannot be recorded without an order.

4.2 Deletion anomalies

- Kiran Hegde's only order was cancelled. If cancelled orders are purged, the customer's name, e-mail and address disappear completely.
- If the only order containing a dish is deleted, the dish, its category and price are lost from the menu.
- Removing a partner's last delivery erases the partner's name, phone and vehicle.

4.3 Update anomalies

- If a restaurant's rating changes from 4.4 to 4.5, every row of every order from that restaurant must be updated; missing one leaves two ratings for one restaurant.
- If a customer changes phone number or becomes a Premium member, all of that customer's rows must be changed.
- If a dish's menu price changes, a careless update would also alter the amount charged in past orders and corrupt revenue history.

5. Functional dependencies between attributes

A functional dependency X -> Y means that the value of X uniquely determines the value of Y.

Functional dependency	Meaning
order_id -> order_date, status, payment_method, delivery_fee, customer_id, restaurant_id, partner_id	An order number identifies the whole order header.
customer_id -> customer_name, email, phone, area, membership	A customer id identifies all customer details.
email -> customer_id ; phone -> customer_id	E-mail and phone are unique per customer (candidate keys).
restaurant_id -> restaurant_name, cuisine, area, rating, is_open	A restaurant id identifies all restaurant details.
partner_id -> partner_name, phone, vehicle, availability	A partner id identifies the partner.
item_id -> item_name, category, price, is_veg, restaurant_id	A dish id identifies the dish and its restaurant.
(order_id, item_id) -> quantity, unit_price	Quantity and price charged depend on both the order and the dish.
(restaurant_name, area) -> restaurant_id	The same restaurant name can exist in different areas (alternate key).

Dependency problems in ORDER_RECORD (key = order_id + item_id):

- **Partial dependencies** (violate 2NF): order_date, status, customer_id and partner_id depend only on *order_id*; item_name, category and price depend only on *item_id* - each is just part of the composite key.
- **Transitive dependencies** (violate 3NF): order_id -> customer_id -> customer_name / membership; order_id -> restaurant_id -> cuisine / rating; order_id -> partner_id -> partner_name. Non-key attributes depend on other non-key attributes.

6. How poor design leads to these anomalies

The flat table stores facts about **several different entities** (customers, restaurants, partners, dishes, orders) in one table. A fact about a customer, dish or partner exists only 'inside' order rows, so it cannot exist without an order (insertion anomaly) and vanishes when the last order is deleted (deletion anomaly). Because those facts are copied into many rows - the direct result of the partial and transitive dependencies above - one real-world change must be made in many places (update anomaly), and any missed copy leaves the data inconsistent.

7. Proposed improvements (normalisation)

- **1NF:** every column holds one atomic value and each row is identified by a key. Satisfied by one row per dish per order (no lists such as 'Dosa, Idli' in one cell).
- **2NF:** split off attributes that depend on part of the key: order header data to *orders*, dish data to *menu_items*; only quantity and price remain in *order_items*.
- **3NF:** remove transitive dependencies: customer data to *customers*, restaurant data to *restaurants*, partner data to *delivery_partners*. Every non-key attribute now depends on 'the key, the whole key and nothing but the key'.

Resulting design (implemented in Assignments 5-9):

Table	Primary key	Foreign keys	Other attributes
customers	customer_id	-	full_name, email, phone, area, signup_date, membership
restaurants	restaurant_id	-	restaurant_name, cuisine, area, rating, is_open
delivery_partners	partner_id	-	partner_name, phone, vehicle, availability
menu_items	item_id	restaurant_id -> restaurants	item_name, category, price, is_veg
orders	order_id	customer_id -> customers, restaurant_id -> restaurants, partner_id -> delivery_partners (nullable)	order_date, status, payment_method, delivery_fee
order_items	order_item_id (unique order_id + item_id)	order_id -> orders, item_id -> menu_items	quantity, unit_price

Note: *order_items.unit_price* deliberately stores the price at the time of the order. This is not harmful redundancy: it is historical data that must not change when the menu price changes later.

Effect of the improvements on the anomalies:

Operation	Before (flat table)	After (normalised)
Add a dish nobody has ordered	Impossible without an order row	Simple INSERT into menu_items
Register a customer with no orders	Impossible	Simple INSERT into customers
Delete a cancelled order	Customer / dish / partner details may be lost	Only order and its lines are removed
Change a restaurant's rating	Update many rows; risk of inconsistency	Update ONE row in restaurants
Customer changes phone number	Update every order row of that customer	Update ONE row in customers
Customer e-mail storage	Repeated on every order line (52 copies)	Stored once per customer (12 rows)

Referential integrity is retained through foreign keys, so the separate tables can always be joined back to reproduce the original flat report (as in the query in Section 2). A further improvement for the future is a separate `addresses` table if a customer can save several delivery addresses.